IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

# Yenepoya Institute of Arts , Science , Commerce & Management

# Project – High Level Design

# on

# Cloud Cost Intelligence Platform for E-commerce

**Student Name:** **AROMAL K S**

**Industry Mentor: Mr.Sumit Kumar** **Guided by: Mr.Praveen**

# Table of Contents


- Introduction
  - Scope of the document.
  - Intended Audience
  - System overview.
- System Design.
  - Application Design
  - Process Flow.
  - Information Flow.
  - Components Design
  - Key Design Considerations
  - API Catalogue.
- Data Design.
  - Data Model
  - Data Access Mechanism
  - Data Retention Policies
  - Data Migration
- Interfaces
- State and Session Management
- Caching
- Non-Functional Requirements
  - Security Aspects
  - Performance Aspects
- References

# INTRODUCTION

This document presents the High Level Design (HLD) of the Cloud Cost Intelligence Platform developed using Python. The system is designed to provide e-commerce enterprises with an automated analytical tool that interacts with cloud billing datasets to provide financial transparency

The Cloud Cost Intelligence Platform is a sophisticated analytical framework designed to bridge the gap between cloud Infrastructure operations and financial accountability in the e-commerce sector. As e-commerce platforms scale, the complexity of cloud billing data increases exponentially, often resulting in "Cloud Sprawl" where resources are underutilized yet fully billed.This project addresses these challenges by utilizing Python and Pandas to ingest, clean, and analyze multi-vendor billing exports. By transforming raw spreadsheets into high-fidelity visual reports, the platform empowers IT managers to perform precise "Cost Management Analysis" and optimize their digital supply chain costs.

## 1.1 Scope of the Document

- The overall System Architecture and the integration of the Jupyter Notebook environment
- Functional behavior of the Pandas-based aggregation logic
- Data handling workflows, specifically focused on processing Excel and CSV billing telemetry
- Key design considerations, including security protocols for handling sensitive financial line items

This document does not include:

- Detailed code-level implementation
- Live Cloud API Integration

## 1.2 Intended Audience

- Industry Mentors: To evaluate the application of Data Analysis skills to IT Management.
- Financial Controllers: To understand the logic behind the cost-saving recommendations.
- Cloud Architects: To analyze the impact of resource allocation on the overall e-commerce budget.

## 1.3 System Overview

●The Cloud Cost Intelligence Platform is a data-driven analytical system developed to provide financial transparency within e-commerce cloud operations. The system operates within a Jupyter Notebook environment, utilizing Python and Pandas to ingest, process, and analyze complex cloud billing datasets (CSV/Excel).

Unlike manual spreadsheet tracking, this platform automates the aggregation of costs, allowing IT managers to identify spending patterns and resource inefficiencies through high-fidelity visual reports.

### Key Functionalities

●Automated Data Ingestion

●Cost Aggregation

●Data Cleaning & Normalization

●Visual Intelligence

●Security & Privacy Masking

The system is designed to be lightweight and stateless; it does not require external database connections and maintains all data in-memory during the analysis runtime to maximize processing speed.

# SYSTEM DESIGN

The Cloud Cost Intelligence Platform is a data-processing system that uses Python and Pandas to analyze e-commerce cloud expenses. It operates locally on a Mac within a Jupyter Notebook environment to ensure data security and fast execution.

**The design ensures**:

● Accurate Totals: Uses Pandas for precise financial calculations.

● Data Privacy: Local processing keeps sensitive billing data secure.

● Simplicity: Operates without complex external databases.

## 2.1 Application Design

The application is divided into logical layers for modularity:

●Input Layer: Reads Excel/CSV billing files using Pandas.

●Processing Layer: Cleans data and handles missing values.

●Logic Layer: Groups costs by Service Type or Month.

●Output Layer: Displays results through Pie Charts.

## 2.2 Process Flow

1. System initializes Jupyter environment.
2. User provides the billing spreadsheet.
3. System validates required columns.
4. Data is converted into a DataFrame.
5. Costs are aggregated by category.
6. Visual Pie Chart is generated.
7. Analysis is displayed to the user.

## 2.3 Information Flow

Raw Excel/CSV Data

↓

Data Loading (Pandas)

↓

Data Preprocessing (Cleaning)

↓

Cost Aggregation Logic

↓

Statistical Output Generation

↓

Final Visualization (Pie Chart Display)

## 2.4 Components Design

1. Data Ingestion Handler

- Captures "Cloud Billing" exports from Excel or CSV files.
- Facilitates the connection between the Mac file system and the Jupyter environment.)

2. Data Cleaning Module

- Scans the dataset for null values or incorrect data types.
- Standardizes currency and date formats for mathematical accuracy.

3. Analytics & Aggregation Engine

- Executes Pandas groupby() and sum() operations
- Calculates total expenditure by service category or billing period.

4. Visualization Generator

- Renders processed data into high-fidelity Pie Charts using Matplotlib.
- Provides a visual breakdown of cloud investment for stakeholders

5. Security Workflow Manager

- Ensures local-only processing to protect sensitive e-commerce data.
- Manages the memory purge policy once the analysis is complete.

## 2.5 Key Design Considerations

- Performance: Uses vectorized operations for instant calculation of large datasets.
- Security: Local-execution isolation prevents external data leakage.
- Scalability: Modular design allows for adding new cloud vendor formats.
- Maintainability: Separation of ingestion and logic layers for easy updates.

## 2.6 API Catalogue

- **Internal APIs:**
- load_data(): Ingests the spreadsheet into the DataFrame.
- process_costs(): Aggregates spend totals.
- show_chart(): Generates the final Pie Chart.
- **External APIs:**

None. The system operates entirely offline for maximum security.

# DATA DESIGN

The data design of the Cloud Cost Intelligence Platform is structured to handle high-volume e-commerce billing telemetry. The system focuses on transforming raw, unstructured spreadsheet data into a standardized format for financial analysis.

## 3.1 Data Model

The platform utilizes an In-Memory Tabular Data Model via the Pandas DataFrame. The logical schema is designed to capture the following essential attributes from the e-commerce cloud provider:

| Attribute Name | Data Type | Description |
|---|---|---|
| Service_Category | String | The specific cloud service (e.g., EC2, S3, RDS) |
| Billing_Month | Date/String | The period for which the cost was incurred |
| Usage_Quantity | Float | The amount of resource consumed |
| Unit_Rate | Float | The cost per unit of the resource |
| Net_Cost | Float | The total calculated cost for that line item |

## 3.2 Data Access Mechanism

The system employs the Pandas Vectorized Read API.

●Direct File Access: The system uses pd.read_excel() and pd.read_csv() to pull data directly into the Mac's RAM.

●No Database Latency: By avoiding traditional SQL databases, the platform achieves near-instantaneous slicing and dicing of hundreds of thousands of rows.

●Vectorization: Instead of processing row-by-row, the system applies calculations across entire columns (vectors) simultaneously, maximizing CPU efficiency on the Mac

### 3.3 Data Retention Policies

● To ensure Security and Data Privacy, the platform follows a Non-Persistent Retention Policy:

● Volatile Storage: Data exists only within the active Jupyter Notebook kernel session.

● Session Termination: Once the user closes the notebook or clears the kernel, all sensitive billing data is purged from the memory.

●Zero Trace: No temporary files containing financial secrets are written to the local disk during processing

### 3.4 Data Migration

●Vendor Portability: The system is designed to handle migration between different e-commerce cloud providers (e.g., switching from AWS to Azure).

●Format Mapping: Migration is achieved by updating the column-mapping dictionary in the Python script to match the new vendor's spreadsheet headers.

●Schema Evolution: Supports the addition of new billing attributes (like "Region" or "Discount Code") without requiring a complete rewrite of the core logic.

# INTERFACES

The interface design for the Cloud Cost Intelligence Platform focuses on seamless interaction between the user, the local file system, and the analytical engine. This section details the touchpoints that facilitate the "Cost Management Analysis" workflow.

## User Interface(ui)

- The primary interface is a web-based interactive dashboard hosted within the Jupyter Notebook environment.It allows the user to execute Python code blocks incrementally to observe data transformations in real-time.

## Input Interface and Output Interface

- **Inbound Interface**: The system utilizes the Pandas I/O API to create a high-speed link with the Mac's local stora

It programmatically identifies and reads .xlsx (Excel) and .csv (Comma Separated Values) file formats.

- **Outbound Interface**: The Matplotlib Rendering Engine acts as the visual output interface, converting raw numeric

- into professional-grade graphical artifacts.

Resulting data summaries can be exported back to the local system as "Cleaned" Excel reports for record-keeping.

-

# STATE & SESSION MANAGEMENT

The Cloud Cost Intelligence Platform operates using a volatile, kernel-based session management system. Because the platform is designed for high-stakes e-commerce financial analysis, state management is optimized for accuracy, security, and computational speed within the Jupyter Notebook environment.

## ●Session Characteristics

● Kernel-Initiated Sessions: A session begins immediately when the Python kernel is initialized within the Jupyter environment. This creates a dedicated memory space on the local Mac for the current analysis.

●Active Runtime Lifespan: The session remains active as long as the notebook kernel is running. This allows the user to perform iterative data slicing and dicing without reloading the raw dataset.

## State Handling Logic

● In-Memory DataFrames: The system maintains a "State" by storing processed e-commerce billing data in Pandas DataFrames. These DataFrames hold the results of cleaning, normalization, and aggregation throughout the session.

●Variable Persistence: All calculated variables, such as total cost sums or service-specific subsets, are stored as active objects in the RAM. This allows for instantaneous cross-referencing between different analytical cells.

## Session Security & Limitations

●Stateless Restart: To ensure a "Security Workflow," restarting the kernel completely clears all stored states. This prevents any carry-over of sensitive financial data between different analysis projects.

●Non-Persistent Storage: No session recovery is possible after the notebook is closed. This is a deliberate design choice to ensure that financial secrets are never written to permanent disk storage during the session.

●Single-User Focus: The current state management is optimized for a single-user local environment, ensuring maximum performance for processing millions of rows of cloud telemetry without multi-user latency.

# CACHING

○The Cloud Cost Intelligence Platform utilizes a multi-tier caching strategy designed to optimize the performance of the Pandas analytical engine during large-scale e-commerce billing processing. By prioritizing in-memory data persistence, the system achieves near-instantaneous query response times for financial stakeholders.

**In-Memory Dataframe Caching**

● Primary Object Caching: Once raw cloud billing data is ingested, it is cached as a Pandas DataFrame object in the Mac's primary memory. This eliminates the need for repeated, expensive I/O operations on the local file system during the session.

● Vectorized Operation Caching: Intermediate calculation results (e.g., grouped service totals or monthly expenditure aggregates) are stored as temporary memory variables. This ensures that visual reports can be refreshed or adjusted without re-executing the entire logic layer.

**Computational Optimization Caching**

● Library Import Caching: The platform leverages Python’s bytecode caching (.pyc) for the Pandas and Matplotlib libraries, reducing the cold-start time of the Jupyter Notebook environment.

● Deterministic Execution: The system caches analytical outputs within the Jupyter Kernel state, maintaining a deterministic environment where repeated code block executions yield consistent, high-speed results.

**Security & Invalidation Protocols**

● Stateless Memory Purge: Following professional FinOps security standards, all cached data exists only in volatile memory (RAM).

● Cache Invalidation: The cache is automatically invalidated and purged upon kernel termination, ensuring no sensitive financial e-commerce secrets are persisted in hidden local storage.

# NON-FUNCTIONAL REQUIREMENTS

- Non-functional requirements (NFRs) define the operational capabilities and constraints of the Cloud Cost Intelligence Platform. These are critical for ensuring that the system is not only functional but also secure, reliable, and high-performing in a professional e-commerce environment.

**Security Aspects**

- The security framework of this platform is designed around the principle of Data Sovereignty, ensuring that sensitive financial e-commerce telemetry never leaves the controlled local environment.
- Local Execution Isolation: The system is architected to run entirely within a local Jupyter Notebook instance on the user's Mac, ensuring an "air-gapped" style of analysis that prevents data leakage to external servers.
- Volatile Data Handling: To protect against unauthorized data recovery, the platform utilizes a Non-Persistent Retention Policy, where all billing data is stored only in volatile RAM and is purged upon session termination.
- Code Integrity: By using standard, open-source libraries like Pandas, the system avoids the security risks associated with unverified third-party "Black Box" analytical software.

**Performance Aspects**

- Performance is a core differentiator for this platform, focusing on the ability to process massive datasets with minimal latency.
- Vectorized Computational Speed: By utilizing Vectorized Operations via the Pandas API, the system can perform complex aggregations on millions of rows of billing data in seconds, far outperforming traditional spreadsheet software.
- In-Memory Efficiency: The system minimizes I/O overhead by loading the entire dataset into a high-speed DataFrame cache, allowing for instantaneous "slice and dice" operations during a live presentation or PPT demo.

## 7.1 Security Aspects

● The system ensures high-level security by operating strictly within a local environment, specifically designed to protect e-commerce financial telemetry.

**Security Features**

● Local Data Sovereignty: All "Cloud Billing" spreadsheets are processed on the local Mac, ensuring no financial data is ever transmitted over the internet.

● No External API Risks: The system does not call external web APIs, eliminating the risk of data interception or MITM attacks.

● In-Memory Volatility: To prevent data theft, sensitive cost info is stored only in RAM and is never written to permanent disk storage.

**Input Handling**

● Standardized Ingestion: User-provided Excel/CSV files are validated for structural integrity before being loaded into the analytical engine.

● Schema Enforcement: Prevents the execution of malicious scripts hidden within data files by strictly enforcing column data types.

**Limitations**

● Single-User Access: Currently lacks multi-user authentication as it is a local standalone tool.

● Physical Security Dependence: Security is dependent on the password protection and encryption of the host Mac hardware.

**Future Enhancements**

● Role-Based Access Control (RBAC): Implementing a login system for different financial team members.

● Data Encryption at Rest: Adding AES-256 encryption for any locally exported cost reports.

## ● PERFORMANCE ASPECTS

The platform is engineered for high-performance data processing, ensuring that complex e-commerce billing analysis is completed with minimal latency and high resource efficiency.

**Efficiency Factors**

● No Database Queries: By utilizing a local in-memory data model, the system avoids the time-consuming input/output (I/O) operations associated with traditional SQL database lookups.

● Zero Network Latency: All analytical computations are performed on the local Mac hardware, eliminating delays caused by network congestion or external server response times.

● Vectorized Execution: Instead of processing billing rows one by one, the engine uses Pandas vectorization to apply calculations across entire columns simultaneously, maximizing CPU efficiency.

**Scalability Considerations**

● Memory-Bound Scaling: Performance is primarily dependent on available system RAM; the system can scale to handle millions of e-commerce records as long as the hardware supports the dataset size.

● Optimization Requirement: While the current architecture is highly efficient, performance may degrade if:

~The volume of cloud billing telemetry exceeds the available physical memory of the Mac.

~Complex multi-dimensional aggregations are performed without utilizing the optimized Pandas groupby()logic.

● Modular Growth: The system is designed to allow for the future integration of parallel processing (using Dask or multiprocessing) if the data scale increases significantly.

# REFERENCES


- Google
- Wikipedia
- ChatGPT , Gemini etc…….